

UNIVERSIDAD AMERICANA



Programación Orientada a Objetos

❖ Desarrollo de una API con Spring Boot

Elaborado por:

Jonathan Rivera Guido

Docente: José Durán García

Fecha de entrega:

13/05/2026

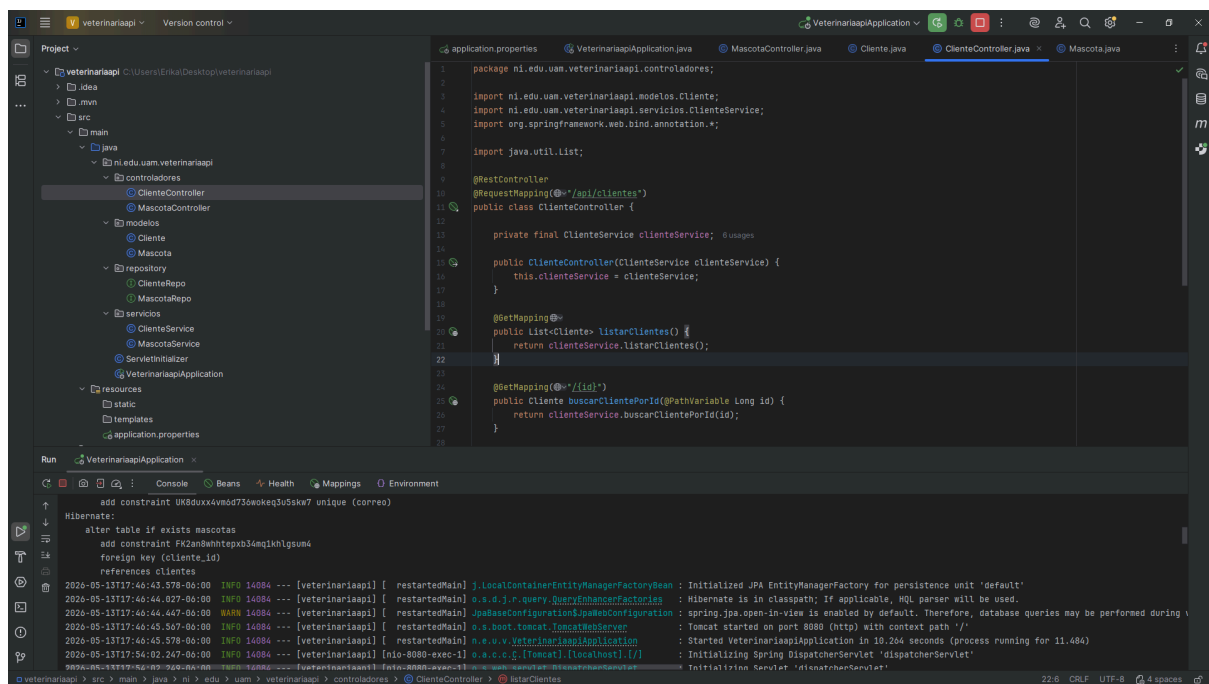
Nombre de programa: Veterinariaapi

- Descripción del proyecto

El presente proyecto consiste en el desarrollo de una API REST elaborada con Spring Boot y Java, orientada a la gestión básica de una veterinaria. La aplicación permite administrar información de clientes y mascotas mediante operaciones CRUD, es decir, crear, consultar, actualizar y eliminar registros. Para cumplir con la relación entre clases, se implementó una asociación entre las entidades Cliente y Mascota, donde un cliente puede tener una o varias mascotas registradas, mientras que cada mascota pertenece a un cliente específico.

La API utiliza JPA/Hibernate para manejar la persistencia de datos y PostgreSQL como base de datos relacional. Además, los endpoints fueron probados mediante Postman, verificando que las operaciones funcionen correctamente y que la relación entre las entidades se refleje en las respuestas del sistema. De esta manera, el proyecto demuestra el uso de una arquitectura básica por capas, compuesta por modelos, repositorios, servicios y controladores.

1. IntelliJ ejecutando el proyecto



2. Creación de cliente

Get:

The screenshot shows the Postman interface with a workspace named "JONATHAN JOSUE RIVERA GUIDO's Workspace". On the left, under "COLLECTIONS", there is a "My Collection" containing several requests, with "GET Get data" selected. The main panel shows a GET request to "http://localhost:8080/api/clientes". The "Body" tab is active, displaying a JSON response:

```
1 [
2   {
3     "id": 3,
4     "nombre": "Jonathan",
5     "apellido": "Rivera",
6     "correo": "jonathan.rivera@uam.edu.ni",
7     "telefono": "8444-0122"
8   }
9 ]
```

The response status is "200 OK" with a response time of 54 ms and a size of 268 B. The "Body" tab also includes options for "Preview", "Visualize", and "Save Response".

Post:

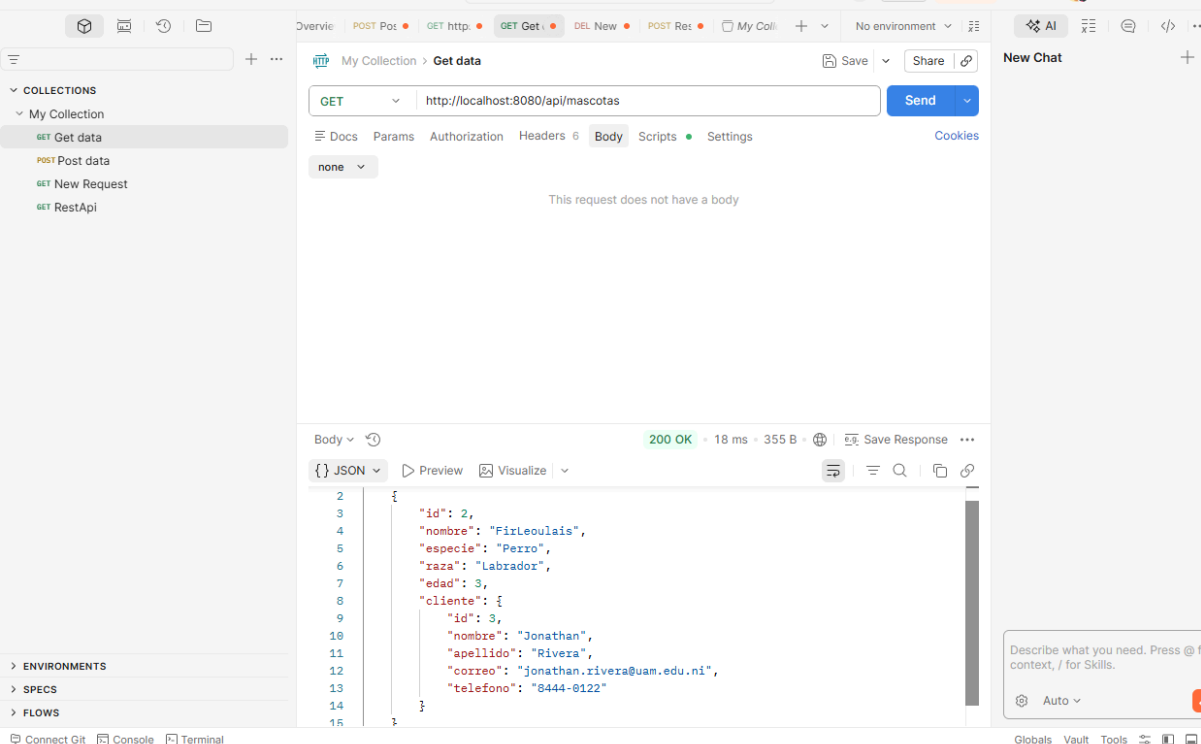
The screenshot shows the Postman interface with the same workspace. The "POST data" request is selected in the "My Collection". The main panel shows a POST request to "http://localhost:8080/api/clientes". The "Body" tab is active, displaying a JSON request body:

```
1 {
2   "nombre": "Jonathan",
3   "apellido": "Rivera",
4   "correo": "jonathan.rivera@uam.edu.ni",
5   "telefono": "8444-0122"
6 }
```

The response status is "200 OK" with a response time of 64 ms and a size of 266 B. The "Body" tab also includes options for "Preview", "Visualize", and "Save Response".

3. Creación de mascotas:

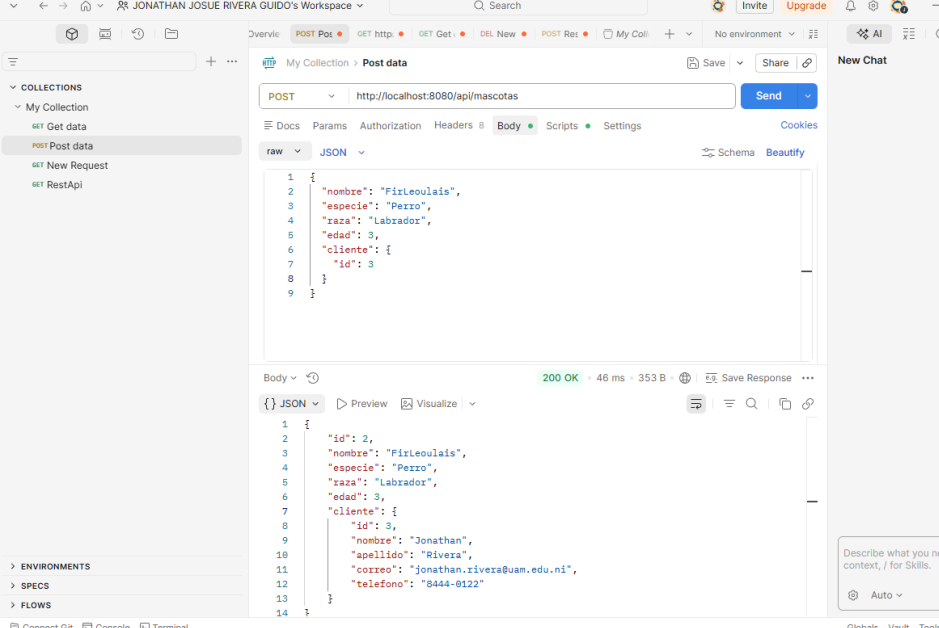
Get:



The screenshot shows the Postman interface with a GET request to `http://localhost:8080/api/mascotas`. The response is a 200 OK status with a response time of 18 ms and a body size of 355 B. The JSON response is as follows:

```
{
  "id": 2,
  "nombre": "Firleoulais",
  "especie": "Perro",
  "raza": "Labrador",
  "edad": 3,
  "cliente": {
    "id": 3,
    "nombre": "Jonathan",
    "apellido": "Rivera",
    "correo": "jonathan.rivera@uam.edu.ni",
    "telefono": "8444-0122"
  }
}
```

Post:



The screenshot shows the Postman interface with a POST request to `http://localhost:8080/api/mascotas`. The request body is a JSON object. The response is a 200 OK status with a response time of 46 ms and a body size of 353 B. The JSON response is as follows:

```
{
  "id": 2,
  "nombre": "Firleoulais",
  "especie": "Perro",
  "raza": "Labrador",
  "edad": 3,
  "cliente": {
    "id": 3,
    "nombre": "Jonathan",
    "apellido": "Rivera",
    "correo": "jonathan.rivera@uam.edu.ni",
    "telefono": "8444-0122"
  }
}
```

Put:

The screenshot displays the REST Client interface. On the left, a sidebar shows a collection named 'My Collection' with several requests, including 'New Request' which is currently selected. The main area is titled 'New Request' and shows a PUT request to the URL 'http://localhost:8080/api/mascotas/2'. The request body is in JSON format, containing a nested object for a client. The response status is '200 OK' with a response time of 55 ms and a size of 345 B. The response body is also in JSON format, containing a nested object for a client with additional details like name, last name, email, and phone number. The bottom of the interface includes a status bar with 'Connect Git', 'Console', and 'Terminal' buttons, and a 'New Chat' button on the right.

Overview POST Pos GET http GET Get PUT New POST Ret My Coll + No environment AI

My Collection > New Request Save Share

PUT http://localhost:8080/api/mascotas/2 Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "nombre": "Leo",
3   "especie": "Perro",
4   "raza": "Labrador",
5   "edad": 3,
6   "cliente": {
7     "id": 3
8   }
9 }
```

Body 200 OK 55 ms 345 B Save Response

{ JSON Preview Visualize

```
1 {
2   "id": 2,
3   "nombre": "Leo",
4   "especie": "Perro",
5   "raza": "Labrador",
6   "edad": 3,
7   "cliente": {
8     "id": 3,
9     "nombre": "Jonathan",
10    "apellido": "Rivera",
11    "correo": "jonathan.rivera@uam.edu.ni",
12    "telefono": "8444-0122"
13  }
14 }
```

ENVIRONMENTS SPECS FLOWS

Connect Git Console Terminal

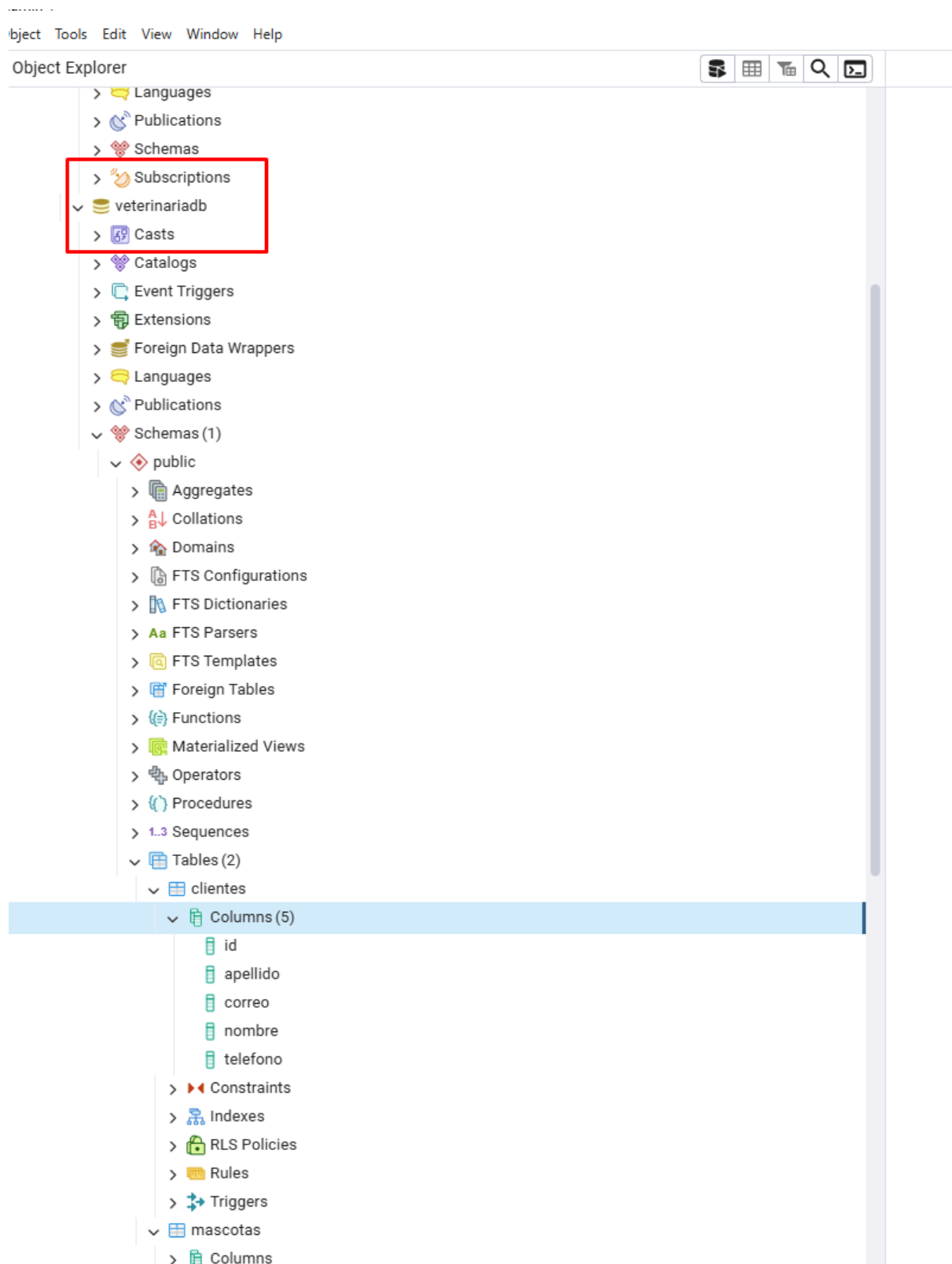
Global Vault Tools

New Chat

Describe what you need context, / for Skills.

Auto

BASE DE DATOS EN PG4 ADMIN:



Link de repositorio: <https://github.com/jriver123/VeterinariaApi.git>